

POSIX Threads

Marc Pérache

`marc.perache@cea.fr`

TDs : David Dureau

2025

Table of Contents

Définitions
Les threads
Exclusion
mutuelle
(mutex)
Sémaphores
Conditions
Volatilité
Clés

- 1 Définitions
- 2 Les threads
- 3 Exclusion mutuelle (mutex)
- 4 Sémaphores
- 5 Conditions
- 6 Volatilité
- 7 Clés

Définitions des Termes Clés

Définitions

Les threads

Exclusion mutuelle (mutex)

Sémaphores

Conditions

Volatilité

Clés

SMP (Symmetric Multiprocessing)

Architecture où plusieurs processeurs partagent une mémoire commune et peuvent exécuter des threads simultanément.

NUMA (Non-Uniform Memory Access)

Architecture où la mémoire est distribuée parmi les processeurs, et l'accès à la mémoire locale est plus rapide que l'accès à la mémoire distante.

Thread

Unité de base d'exécution dans un processus, partageant le même espace mémoire que les autres threads du même processus.

Définitions des Termes Clés

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Processus

Instance d'un programme en cours d'exécution, avec son propre espace mémoire et ses propres ressources.

Préemption

Interruption forcée d'un thread ou processus en cours d'exécution pour permettre à un autre thread ou processus de s'exécuter.

Section Critique

Partie du code où l'accès à une ressource partagée doit être protégé pour éviter les conditions de course.

Définitions des Termes Clés

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

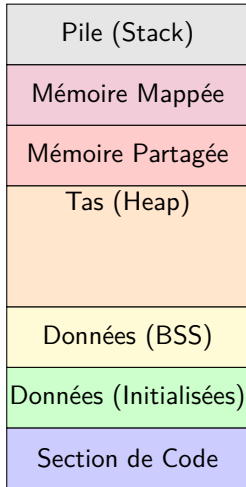
Attente Active (Busy Waiting)

Technique où un thread attend une condition en vérifiant continuellement une variable ou une condition.

Réentrance

Propriété d'une fonction qui peut être appelée de manière sûre par plusieurs threads simultanément.

Structure Mémoire d'un Processus UNIX



Section de Code : Contient le code exécutable du programme.

Données (Initialisées) : Contient les variables globales et statiques initialisées.

Données (BSS) : Contient les variables globales et statiques non initialisées.

Tas (Heap) : Utilisé pour l'allocation dynamique de mémoire pendant l'exécution du programme.

Mémoire Partagée : Utilisée pour la communication inter-processus (IPC).

Mémoire Mappée : Utilisée pour mapper des fichiers ou des périphériques directement dans l'espace d'adressage du processus.

Pile (Stack) : Utilisée pour stocker les variables locales, les paramètres de fonction, et les adresses de retour.

Définitions

Les threads

Exclusion mutuelle (mutex)

Sémaphores

Conditions

Volatilité

Clés

Structure Mémoire d'un Processus Multithread

Définitions

Les threads

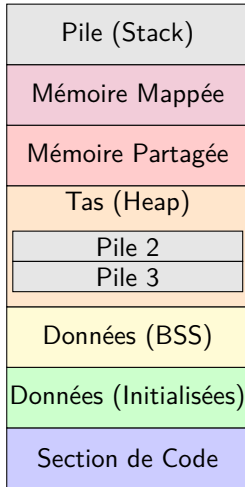
Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés



Section de Code : Partagée entre tous les threads.

Données (Initialisées) : Partagées entre tous les threads.

Données (BSS) : Partagées entre tous les threads.

Tas (Heap) : Partagé entre tous les threads.

Mémoire Partagée : Partagée entre tous les threads.

Mémoire Mappée : Partagée entre tous les threads.

Pile (Stack) : Chaque thread a sa propre pile pour les variables locales, les paramètres de fonction, et les adresses de retour.

Programmation avec Mémoire Distribuée

Définitions

Les threads

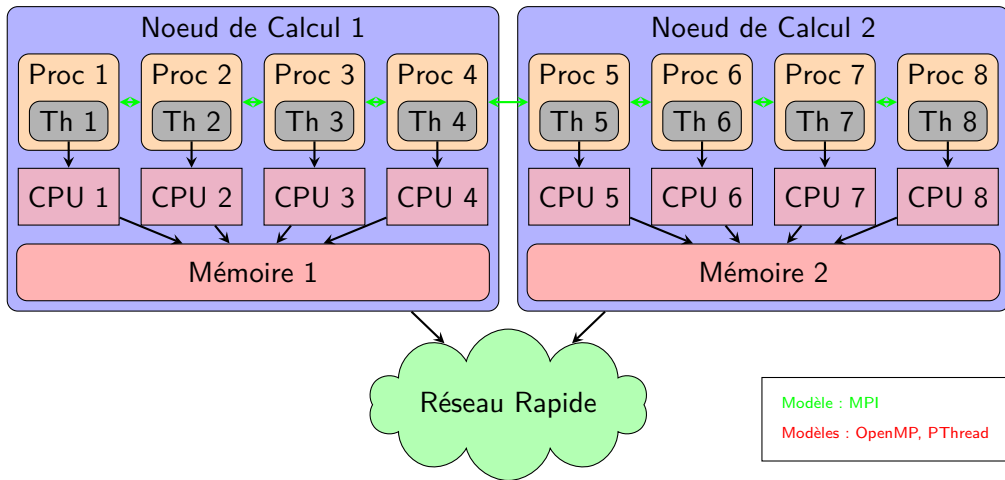
Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés



Programmation avec Mémoire Partagée

Définitions

Les threads

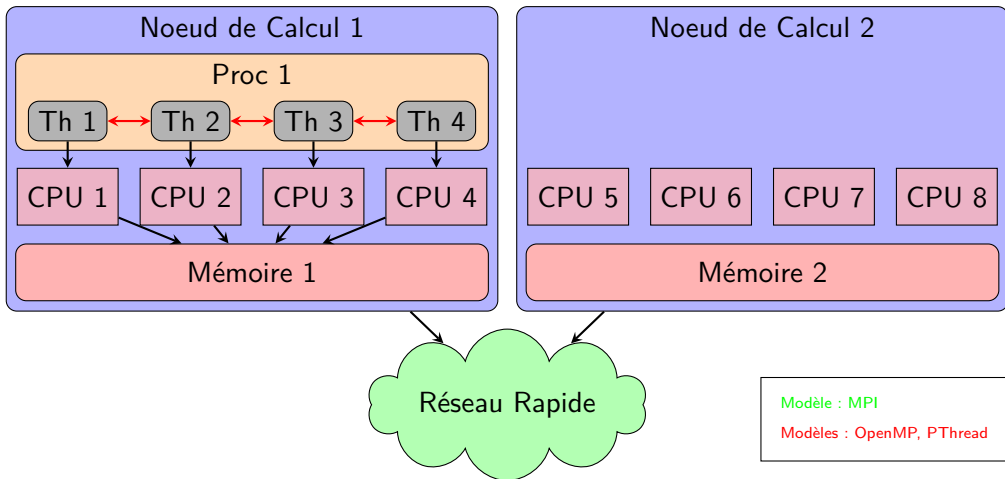
Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés



Programmation Hybride

Définitions

Les threads

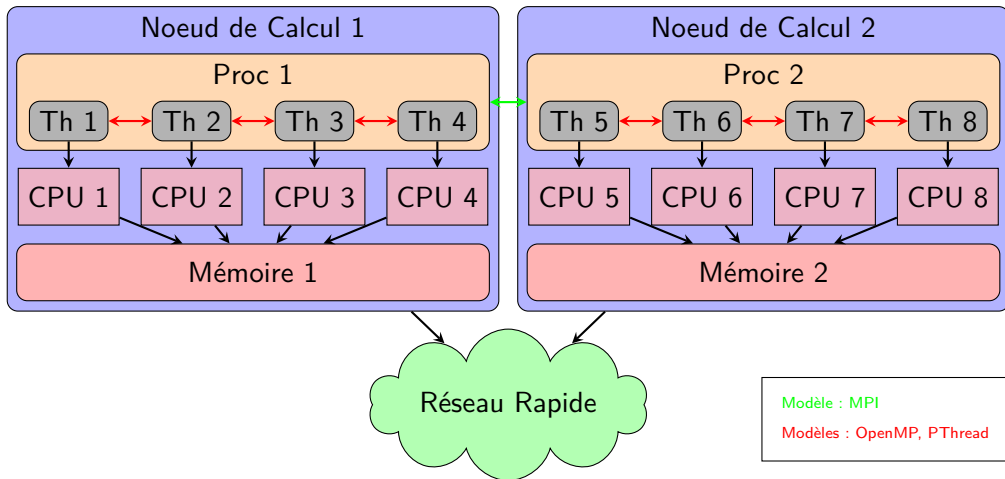
Exclusion mutuelle (mutex)

Sémaphores

Conditions

Volatilité

Clés



Programmation Hybride

Définitions

Les threads

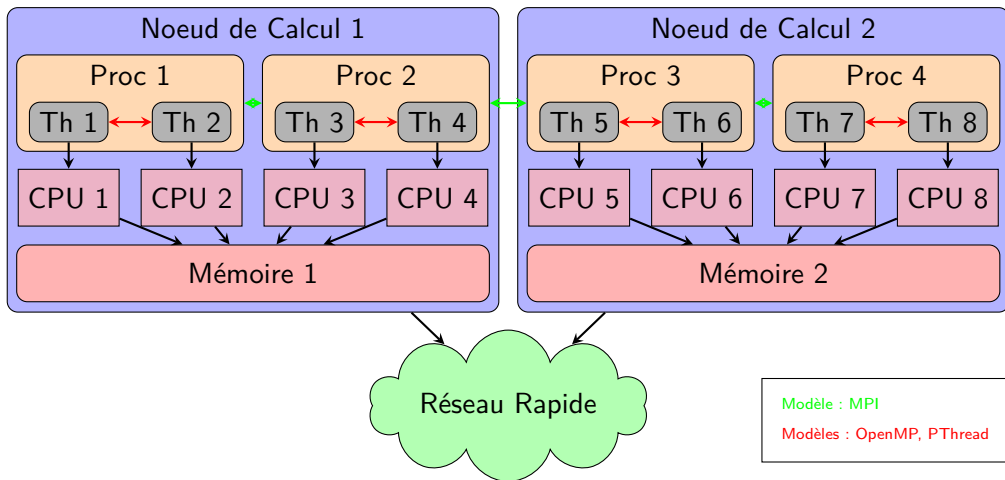
Exclusion mutuelle (mutex)

Sémaphores

Conditions

Volatilité

Clés



- **Qu'est-ce qu'un thread ?**

- Unité de base d'exécution dans un processus.
- Partage le même espace mémoire que les autres threads du même processus.
- Contrairement aux processus, les threads partagent le même contexte d'exécution (mémoire, fichiers ouverts, etc.).

- **POSIX Threads (Pthreads)**

- Standard pour la programmation de threads en C/C++.
- Portable et largement utilisé sur différents systèmes d'exploitation (Linux, Unix, macOS, etc.).
- Fournit une API riche pour la création, la gestion et la synchronisation des threads.

Avantages des Threads

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Parallélisme**

- Permet l'exécution simultanée de plusieurs tâches.
- Améliore les performances des applications en exploitant les processeurs multi-curs.

- **Partage de ressources**

- Les threads partagent la mémoire et les ressources du processus, ce qui réduit la surcharge liée à la communication inter-processus.
- Facilite le partage de données entre les threads.

Avantages des Threads

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Réactivité**
 - Améliore la réactivité des applications en permettant l'exécution de tâches en arrière-plan sans bloquer l'interface utilisateur.
 - Permet de gérer plusieurs tâches simultanément, comme la lecture de données et le traitement de ces données.
- **Efficacité**
 - Réduit la surcharge liée à la création et à la gestion des processus.
 - Les threads sont plus légers que les processus, ce qui permet de créer un plus grand nombre de threads avec moins de ressources.

Définitions des Fonctions POSIX pour les Threads

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **pthread_create**

```
1 int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
    void *(*start_routine) (void *), void *arg);
```

- Crée un nouveau thread qui commence son exécution en appelant `start_routine`.
- **Paramètres :**
 - `thread` : Pointeur vers un objet `pthread_t` où l'identifiant du nouveau thread sera stocké.
 - `attr` : Pointeur vers un objet `pthread_attr_t` qui spécifie les attributs du thread. Peut être NULL pour utiliser les attributs par défaut.
 - `start_routine` : Pointeur vers la fonction que le nouveau thread exécutera.
 - `arg` : Pointeur vers un argument à passer à `start_routine`.
- **Valeur de retour :** Renvoie 0 en cas de succès, ou un code d'erreur en cas d'échec.

Définitions des Fonctions POSIX pour les Threads

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **pthread_join**

```
1 int pthread_join(pthread_t thread, void **retval);
```

- Attend la terminaison du thread spécifié et récupère sa valeur de retour.
- **Paramètres :**
 - **thread** : Identifiant du thread à attendre.
 - **retval** : Pointeur vers un pointeur où la valeur de retour du thread sera stockée. Peut être NULL si la valeur de retour n'est pas nécessaire.
- **Valeur de retour** : Renvoie 0 en cas de succès, ou un code d'erreur en cas d'échec.

Définitions des Fonctions POSIX pour les Threads

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **pthread_exit**

```
1 void pthread_exit(void *retval);
```

- Termine le thread appelant et renvoie une valeur de retour.
- **Paramètres :**
 - `retval` : Pointeur vers la valeur de retour du thread. Peut être NULL.
- **Valeur de retour :** Cette fonction ne retourne pas au thread appelant.

- **pthread_kill**

```
1 int pthread_kill(pthread_t thread, int sig);
```

- Envoie un signal au thread spécifié.
- **Paramètres :**
 - `thread` : Identifiant du thread auquel le signal sera envoyé.
 - `sig` : Le signal à envoyer.
- **Valeur de retour :** Renvoie 0 en cas de succès, ou un code d'erreur en cas d'échec.

Exemple de Code

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 void* thread_function(void* arg) {
6     printf("Hello from thread!\n");
7     return NULL;
8 }
9
10 int main() {
11     pthread_t thread;
12     if (pthread_create(&thread,
13                        NULL, thread_function, NULL)) {
14
15         fprintf(stderr, "Error
16         creating thread\n");
17         return 1;
18     }
19     if (pthread_join(thread, NULL)) {
20
21         fprintf(stderr, "Error
22         joining thread\n");
23         return 2;
24     }
25     printf("Thread finished\n");
26     return 0;
27 }

```

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Commande de compilation :**

```
1 gcc -pthread -o thread_example thread_example.c
```

- **Exécution :**

```
1 ./thread_example
```

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Vérification des retours de fonctions :**
 - Toujours vérifier les valeurs de retour des fonctions Pthreads.
- **Messages d'erreur :**
 - Utiliser perror ou strerror pour afficher des messages d'erreur clairs.

Introduction aux Mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Mutex (Mutual Exclusion)

Mécanisme de synchronisation utilisé pour contrôler l'accès à une ressource partagée, assurant qu'une seule thread peut accéder à la ressource à la fois.

- **Qu'est-ce qu'un mutex ?**
 - Mutex = Mutual Exclusion (exclusion mutuelle).
 - Mécanisme de synchronisation utilisé pour contrôler l'accès à une ressource partagée.
 - Assure qu'une seule thread peut accéder à la ressource à la fois.
- **Pourquoi utiliser un mutex ?**
 - Prévient les conditions de course (race conditions).
 - Garantit la cohérence des données partagées entre les threads.

Fonctions Principales des Mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Initialisation**

- `pthread_mutex_init` : Initialise un mutex.
- Exemple : `pthread_mutex_init(&mutex, NULL);`
- **Initialisation statique**
- Exemple : `pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;`

- **Verrouillage**

- `pthread_mutex_lock` : Verrouille le mutex.
- Exemple : `pthread_mutex_lock(&mutex);`

- **Deverrouillage**

- `pthread_mutex_unlock` : Déverrouille le mutex.
- Exemple : `pthread_mutex_unlock(&mutex);`

- **Destruction**

- `pthread_mutex_destroy` : Détruit un mutex.
- Exemple : `pthread_mutex_destroy(&mutex);`

Exemple de Code avec Mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

```

1 #include <pthread.h> 17
2 #include <stdio.h> 18
3 #include <stdlib.h>
4 19
5 pthread_mutex_t mutex; 20
6 int shared_data = 0; 21
7
8 void* thread_function(void* arg) {
9     pthread_mutex_lock(&mutex); 22
10    shared_data++; 23
11    printf("Thread %ld : shared_data 24
    = %d\n", (long)arg, shared_data 25
    );
12    pthread_mutex_unlock(&mutex); 26
13    return NULL; 27
14 } 28
15 29
16 int main() { 30 }

pthread_t threads[2];
pthread_mutex_init(&mutex, NULL
);

for (long i = 0; i < 2; i++) {
    pthread_create(&threads[i],
        NULL, thread_function, (void*)i
    );
}

for (int i = 0; i < 2; i++) {
    pthread_join(threads[i],
        NULL);
}

pthread_mutex_destroy(&mutex);
return 0;

```

- **Vérification des retours de fonctions**

- Toujours vérifier les valeurs de retour des fonctions de mutex.
- Exemple :

```
1 if (pthread_mutex_lock(&mutex) != 0) {
2     perror("pthread_mutex_lock");
3     exit(EXIT_FAILURE);
4 }
```

- **Messages d'erreur**

- Utiliser `perror` ou `strerror` pour afficher des messages d'erreur clairs.

Conclusion sur les Mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Importance des mutex**

- Essentiels pour la synchronisation des threads.
- Préviennent les conditions de course et assurent la cohérence des données.

- **Bonnes pratiques**

- Toujours verrouiller un mutex avant d'accéder à une ressource partagée.
- Déverrouiller le mutex dès que possible pour éviter les blocages.

- **Documentation**

- Consulter la documentation officielle pour plus de détails sur les mutex et leurs fonctions.

Anatomie d'un mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Structure d'un mutex ainsi que d'un slot pour gérer la liste des threads en attente

```

1 typedef struct slot_s {
2     thread_t *thread;
3     struct slot_s *next;
4 } slot_t;
5
6 typedef struct {
7     /* Compteur de threads dans ou en attente de section critique */
8     volatile int nb_threads;
9     /* Liste des thread bloqués */
10    volatile slot_t *list_first;
11    volatile slot_t *list_last;
12    /* Spinlock pour verrouiller les accès aux données du mutex */
13    spinlock_t lock;
14 } mutex_t;
    
```

Anatomie d'un mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Fonction de verrouillage : lock

```

1 void mutex_lock (mutex_t * m)
2 {
3     slot_t slot;
4     spinlock (&(m->lock));
5     if (m->nb_thread == 0) {
6         m->nb_thread = 1;
7         spinunlock (&(m->lock));
8     } else {
9         slot.thread = thread_self ();
10        enqueue (&slot, m);
11        thread_self ()->status = blocked;
12        register_spinunlock (&(m->lock));
13        yield ();
14    }
15 }

```

Anatomie d'un mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Fonction de déverrouillage : unlock

```

1 void mutex_unlock (mutex_t * m)
2 {
3     slot_t *slot;
4     spinlock (&(m->lock));
5     if (m->list_first != NULL) {
6         slot = dequeue (m);
7         wake (slot->thread);
8     } else {
9         m->nb_thread = 0;
10    }
11    spinunlock (&(m->lock));
12 }
```

Anatomie d'un mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Fonction de test : trylock

```
1 int mutex_trylock (mutex_t * m)
2 {
3     slot_t slot;
4     spinlock (&(m->lock));
5     if (m->nb_thread == 0) {
6         m->nb_thread = 1;
7         spinunlock (&(m->lock));
8         return 0;
9     }
10    spinunlock (&(m->lock));
11    return 1;
12 }
```

Anatomie d'un mutex

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Fonction de test : trylock

```

1 int mutex_trylock (mutex_t * m)
2 {
3     slot_t slot;
4     int res = 1;
5     spinlock (&(m->lock));
6     if (m->nb_thread == 0) {
7         m->nb_thread = 1;
8         res = 0;
9         goto fin;
10    }
11    fin :
12    spinunlock (&(m->lock));
13    return res;
14 }
```

Anatomie d'un mutex récursif

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Fonction de verrouillage : lock

```

1 void mutex_lock (mutex_t * m) {
2     slot_t slot;
3     spinlock (&(m->lock));
4     if (m->nb_thread == 0) {
5         m->nb_thread = 1;
6         m->owner = thread_self ();
7         spinunlock (&(m->lock));
8     } else {
9         if (m->owner == thread_self ())
10            {
11                m->step++;
12                spinunlock (&(m->lock));
13            } else {
14                slot.thread = thread_self ();
15                enqueue (&slot, m);
16                thread_self ()->status =
17                    blocked;
18                register_spinunlock (&(m->
19                    lock));
20                yield ();
21            }
22    }
23 }
```

Anatomie d'un mutex récursif

Fonction de déverrouillage : unlock

```

1 void mutex_unlock (mutex_t * m) {
2     slot_t *slot;
3     spinlock (&(m->lock));
4     if (m->step == 1) {
5         m->step--;
6         if (m->list_first != NULL) {
7             slot = dequeue (m);
8             wake (slot->thread);
9         } else {
10            m->nb_thread = 0;
11        }
12    } else {
13        m->step--;
14    }
15    spinunlock (&(m->lock));
16 }

```

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Anatomie d'un mutex récursif

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Fonction de test : trylock

```

1 int mutex_trylock (mutex_t * m){
2     slot_t slot;
3     spinlock (&(m->lock));
4     if (m->nb_thread == 0) {
5         m->nb_thread = 1;
6         m->owner = thread_self ();
7         spinunlock (&(m->lock));
8         return 0;
9     } else {
10        if (m->owner == thread_self ())
11            {
12                m->step++;
13                spinunlock (&(m->lock));
14                return 0;
15            }
16        spinunlock (&(m->lock));
17        return 1;
18    }

```

Sémaphore - Définition

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Sémaphore

Mécanisme de synchronisation utilisé pour contrôler l'accès à une ressource partagée, permettant de gérer un nombre limité de threads accédant simultanément à la ressource.

Opérations sur les sémaphores :

- Incrémenter le compteur
- Décrémenter le compteur

Si l'on essaye de décrémenter un compteur déjà à zéro, alors le thread est bloqué. Pour utiliser les sémaphores, il faut inclure `semaphore.h`.

Sémaphore - Fonctions

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Initialisation d'un sémaphore :

```
int sem_init(sem_t *sem, int pshared, unsigned int val);
```

- Sémaphore non nommé
- L'argument `val` spécifie la valeur initiale du sémaphore.
- L'argument `pshared` indique si ce sémaphore sera partagé entre les threads d'un processus ou entre processus.

Destruction d'un sémaphore :

```
int sem_destroy(sem_t *sem);
```

Décrémentation (verrouillage) d'un sémaphore :

```
int sem_wait(sem_t *sem) ;
```

```
int sem_wait(sem_t *sem);
```

- Si la valeur du sémaphore est plus grande que 0, la décrémentation s'effectue et la fonction revient immédiatement.
- Si le sémaphore vaut zéro, l'appel bloquera jusqu'à ce que le sémaphore devienne disponible pour effectuer la décrémentation (c'est-à-dire que la valeur du sémaphore n'est plus nulle).

Test de verrouillage :

```
int sem_trywait(sem_t *sem) ;
```

- Si la décrémentation ne peut pas être effectuée immédiatement, l'appel renvoie une erreur.
- La variable `errno` vaut `EAGAIN` (l'appel n'est alors pas bloquant).

Incrémentation (déverrouillage) d'un sémaphore :

```
int sem_post(sem_t *sem) ;
```

- Si, à la suite de cet incrément, la valeur du sémaphore devient supérieure à zéro, un autre processus ou thread bloqué dans un appel `sem_wait` sera réveillé et procédera au verrouillage du sémaphore.

Conditions - Définition

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Conditions (Condition Variables)

Mécanisme de synchronisation utilisé pour gérer les conditions d'attente entre les threads, permettant à une thread d'attendre qu'une condition soit remplie avant de continuer.

Opérations fondamentales sur les conditions :

- Signaler la condition (quand le prédicat devient vrai)
- Attendre la condition en suspendant l'exécution du thread jusqu'à ce qu'un autre thread signale la condition

Une variable condition doit toujours être associée à un mutex, pour éviter une condition concurrente où un thread se prépare à attendre une condition et un autre thread signale la condition juste avant que le premier n'attende réellement.

Conditions - Fonctions

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Initialisation d'une variable condition :

```
int pthread_cond_init(pthread_cond_t *cond,
    pthread_condattr_t *cond_attr);
```

- Utilisation des attributs par défaut si `cond_attr` vaut `NULL`.

Initialisation statique :

- Utilisation de la constante `PTHREAD_COND_INITIALIZER`.

Relancer un des threads attendant une variable condition :

```
int pthread_cond_signal(pthread_cond_t *cond);
```

- S'il n'existe aucun thread répondant à ce critère, rien ne se produit.
- Si plusieurs threads attendent sur `cond`, seul l'un d'entre eux sera relancé, mais il est impossible de savoir lequel.

Conditions - Fonctions

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

Relancer tous les threads attendant sur une variable condition :

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

- Rien ne se passe s'il n'y a aucun thread attendant sur cond.

Déverrouiller atomiquement le mutex et attendre qu'une variable condition soit signalée :

```
int pthread_cond_wait(pthread_cond_t *cond,  
pthread_mutex_t *mutex);
```

- L'exécution du thread est suspendue et ne consomme pas de temps CPU jusqu'à ce que la variable condition soit signalée.
- Le mutex doit être verrouillé par le thread appelant à l'entrée de `pthread_cond_wait()`.
- Avant de rendre la main au thread appelant, `pthread_cond_wait()` reverrouille le mutex.

Déverrouillage du mutex et suspension de l'exécution :

- Le déverrouillage du mutex et la suspension de l'exécution sur la variable condition sont liés atomiquement. Si tous les threads verrouillent le mutex avant de signaler la condition, il est garanti que la condition ne peut être signalée (et donc ignorée) entre le moment où un thread verrouille le mutex et le moment où il attend sur la variable condition.

Destruction d'une variable condition :

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

- Libération des ressources qu'elle possède.
- Aucun thread ne doit attendre sur la condition à l'entrée de `pthread_cond_destroy()`.

Exercice

- Comment écrire des sémaphores avec des mutex et des conditions ?

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <assert.h>
4
5 typedef struct{
6 } m_semaphore_t;
7
8 int m_sem_init(m_semaphore_t* sem,
9               unsigned int val){
10     return 0;
11 }
12 int m_sem_wait(m_semaphore_t* sem){
13     return 0;
14 }
15
16 int m_sem_post(m_semaphore_t *sem){
17
18     return 0;
19 }
20
21
22 int main(){
23     return 0;
24 }

```

Exercice

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Comment écrire des sémaphores avec des mutex et des conditions ?

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <assert.h>
4
5 typedef struct {
6     pthread_mutex_t mutex;
7     pthread_cond_t cond;
8     volatile unsigned int value;
9 } m_semaphore_t;
10
11 int m_sem_init(m_semaphore_t* sem,
```

```

    unsigned int val){
12     pthread_cond_t cond =
        PTHREAD_COND_INITIALIZER;
13     pthread_mutex_t mut =
        PTHREAD_MUTEX_INITIALIZER;
14     sem->mutex = mut;
15     sem->cond = cond;
16     sem->value = val;
17     return 0;
18 }
```

Exercice

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Comment écrire des sémaphores avec des mutex et des conditions ?

```

1 int m_sem_wait(m_semaphore_t* sem){ 9 }
2 pthread_mutex_lock(&(sem->mutex)) 10
   ;
3 if(sem->value == 0){
4   pthread_cond_wait(&(sem->cond)
   ,&(sem->mutex)) ;
5 }
6 assert(sem->value > 0);
7 sem->value --;
8 pthread_mutex_unlock(&(sem->mutex
   ));

11 int m_sem_post(m_semaphore_t *sem){
12   pthread_mutex_lock(&(sem->mutex))
   ;
13   sem->value ++;
14   pthread_cond_signal(&(sem->cond))
   ;
15   pthread_mutex_unlock(&(sem->mutex
   ));
16 }
```

Introduction au Mot-Clé volatile

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Définition**

Volatile

Mot-clé en C/C++ indiquant au compilateur que la valeur d'une variable peut être modifiée à tout moment par des opérations externes au flux de contrôle actuel, empêchant ainsi certaines optimisations.

- **Utilité**

- Prévient les optimisations incorrectes du compilateur.
- Essentiel pour les variables partagées entre threads ou modifiées par des interruptions matérielles.

Exemple en Assembleur x86

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

• Objectif

- Simuler une interruption matérielle qui modifie une variable partagée.
- Montrer l'importance de lire la variable à chaque itération.

• Code Assembleur

```

1 wait_loop :                               terminer le programme
2     ; Lire la variable partagée           12     mov eax, 1                ; syscall
3     mov al, [shared_var]                  : exit
4     ; Vérifier si la variable a été      13     xor ebx, ebx            ; status :
    ;   modifiée par l'                     0
    ;   interruption                       14     int 0x80            ; appeler
    ;                                       le noyau
5     cmp al, 1
6     je variable_modified                  15
7     ; Boucler jusqu'à ce que la          16 ; Simuler une interruption maté
    ;   variable soit modifiée              rielle qui modifie la
    ;                                       variable partagée
8     jmp wait_loop
9
10 variable_modified :                       17 interrupt_handler :
11     ; La variable a été modifiée,        18     mov byte [shared_var], 1
    ;                                       19     iret

```

Explication du Code Assembleur

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Variable Partagée**

- La variable `shared_var` est initialisée à 0.

- **Boucle d'Attente Active**

- Le programme entre dans une boucle d'attente active (`wait_loop`) où il lit continuellement la variable `shared_var` jusqu'à ce qu'elle soit modifiée par une interruption.

- **Interruption Matérielle**

- L'interruption matérielle simulée (`interrupt_handler`) modifie la variable `shared_var` en la mettant à 1.

- **Sortie de la Boucle**

- Lorsque la variable est modifiée, le programme sort de la boucle et se termine.

Exemple en C/C++

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Objectif**

- Montrer l'utilité du mot-clé `volatile` pour garantir que le compilateur ne fait pas d'optimisations incorrectes.

- **Code C/C++**

```
1 #include <stdio.h>
2
3 volatile int shared_var = 0;
4 void interrupt_handler() {
5     shared_var = 1;
6 }
7
8 int main() {
9     // Initialiser la variable
10    // partagée
11    shared_var = 0;
12    // Simuler une boucle d'
```

```
attente active
13 while (shared_var == 0) {
14     // Attendre que la
15     // variable soit modifiée par l'
16     // interruption
17 }
18 // La variable a été modifiée
19 // , terminer le programme
20 printf("Variable modified !\n"
21 );
22 return 0;
23 }
```

Explication du Code C/C++

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **Variable Partagée**

- La variable `shared_var` est déclarée comme `volatile` pour indiquer au compilateur qu'elle peut être modifiée à tout moment.

- **Boucle d'Attente Active**

- Le programme entre dans une boucle d'attente active où il lit continuellement la variable `shared_var` jusqu'à ce qu'elle soit modifiée par une interruption.

- **Interruption Matérielle**

- La fonction `interrupt_handler` simule une interruption matérielle qui modifie la variable `shared_var` en la mettant à 1.

- **Sortie de la Boucle**

- Lorsque la variable est modifiée, le programme sort de la boucle et affiche un message.

- **Importance du Mot-Clé volatile**

- Essentiel pour garantir que le compilateur ne fait pas d'optimisations incorrectes.
- Utilisé pour les variables partagées entre threads ou modifiées par des interruptions matérielles.

- **Bonnes Pratiques**

- Toujours utiliser `volatile` pour les variables partagées ou modifiées par des opérations externes.
- Vérifier les valeurs de retour des fonctions et utiliser des mécanismes de synchronisation appropriés.

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Programmation concurrente et gestion des données spécifiques à un thread.
- Trois approches en C et C++ :
 - Clés POSIX Thread (pthread key)
 - Le mot-clé `thread_local` en C++
 - L'attribut `__thread` en C

Clés POSIX Thread (C et C++)

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Permettent d'associer des données spécifiques à chaque thread.
- API :
 - `pthread_key_create`
 - `pthread_setspecific`
 - `pthread_getspecific`
 - `pthread_key_delete`
- Exemple :

```
1 pthread_key_t key;
2 pthread_key_create(&key, NULL);
3 pthread_setspecific(key, valeur);
4 void *val = pthread_getspecific(key);
5 pthread_key_delete(key);
```

Le mot-clé `thread_local` (C++)

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Introduit en C++11 pour gérer des variables locales à un thread.
- Plus simple et performant que les clés POSIX Thread.
- Utilisation :

```
1 thread_local int compteur = 0;
2 void fonction() {
3     compteur++;
4 }
```

L'attribut `__thread` (C)

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Extension GNU C pour définir des variables locales à un thread.
- Moins flexible que les clés POSIX mais plus performant.
- Utilisation :

```
1 __thread int compteur = 0;
2 void fonction() {
3     compteur++;
4 }
```

Comparaison des trois approches

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- **POSIX Thread Key** (C et C++) :
 - Compatible avec les deux langages.
 - Plus flexible mais nécessite des appels système.
- **thread_local** (C++ uniquement) :
 - Plus simple et performant.
 - Limité aux types de données statiques.
- **__thread** (C uniquement) :
 - Performant et simple.
 - Moins portable que POSIX Threads.

Définitions

Les threads

Exclusion
mutuelle
(mutex)

Sémaphores

Conditions

Volatilité

Clés

- Les clés POSIX sont utiles pour des environnements multi-langages et des besoins plus complexes.
- `thread_local` est à privilégier en C++ moderne pour sa simplicité et performance.
- `__thread` est une alternative efficace en C lorsqu'elle est supportée.